

A. Introduction & Objective

This document details the analysis of the *anon_jobs.db3* database, which contains the GWA-T-4 AuverGrid workload trace. This trace is a real-world log of 404,176 jobs from a scientific computing grid in France.

Our objective is to perform a workload characterization, a common task in computer systems research. We aim to answer two key questions:

1. **What is the nature of the jobs?** Are they long or short? Do users have to wait long for them to start?
2. **How efficiently are resources used?** Specifically, are users "wasteful" by requesting far more memory than their jobs actually use?

B. Data Loading and Inspection

The file *anon_jobs.db3* is an SQLite database. The first step is to connect to it and load the data. We discovered the data is stored in a table named Jobs.

We load this table into a pandas DataFrame and immediately inspect it. The key columns for our analysis are:

- **RunTime:** The job's actual execution time in seconds.
- **WaitTime:** The time in seconds a job spent in the queue.
- **ReqMemory:** The amount of memory the user requested (in KB). A value of -1 means "not specified."
- **UsedMemory:** The actual peak memory the job used (in KB).

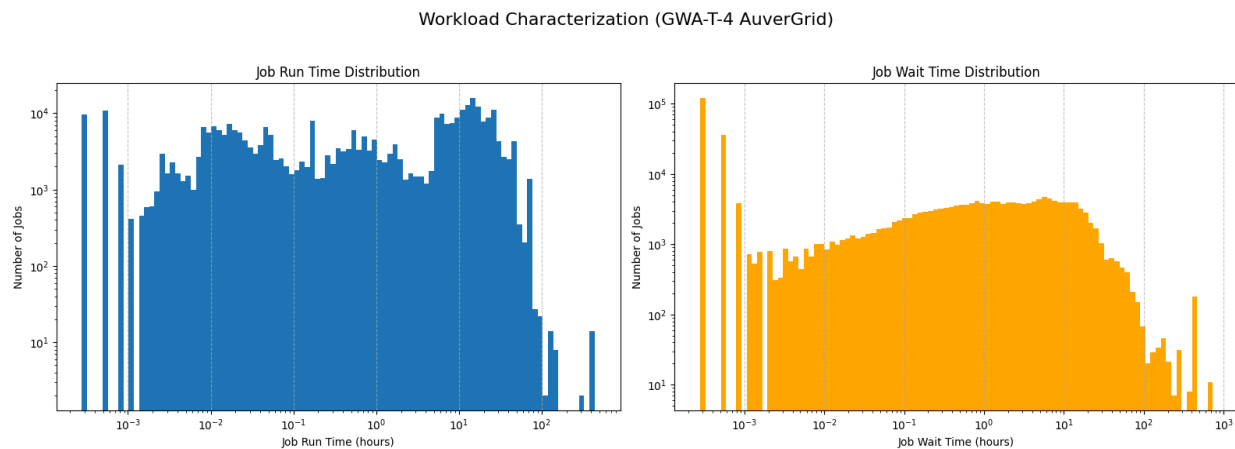
C. Analysis 1: Job Runtime and Wait Time

To understand the system's behavior, we plot the distribution of job runtimes and wait times.

Code Action: We convert RunTime and WaitTime from seconds to hours to make the graphs more human-readable. We then plot two histograms.

Critical Technique (Log-Log Scale): The job times span a massive range (from seconds to many days). A normal plot would just show a single bar on the left. We use a logarithmic scale

for both the X-axis (time) and Y-axis (number of jobs). This lets us see the shape of the distribution for both very short and very long jobs.



Interpreting the Plot: "Workload Characterization"

The first plot, *gwa_t4_analysis_1_runtimes_waittimes.png*, shows:

- **Job Run Time (Left Plot):** This graph shows a classic "heavy-tailed" distribution.
 - **Insight:** The vast majority of jobs are very short (less than a few hours). However, there is a long "tail" of jobs that run for hundreds of hours. This is typical of a scientific grid: many small test/setup jobs, and a few massive, multi-day simulations.
- **Job Wait Time (Right Plot):** This graph shows how long jobs waited in the queue.
 - **Insight:** You can see a large spike on the far left, meaning many jobs start almost instantly (less than an hour wait). However, there's another significant "hump" of jobs that had to wait for several hours or even days, indicating the cluster was often busy.

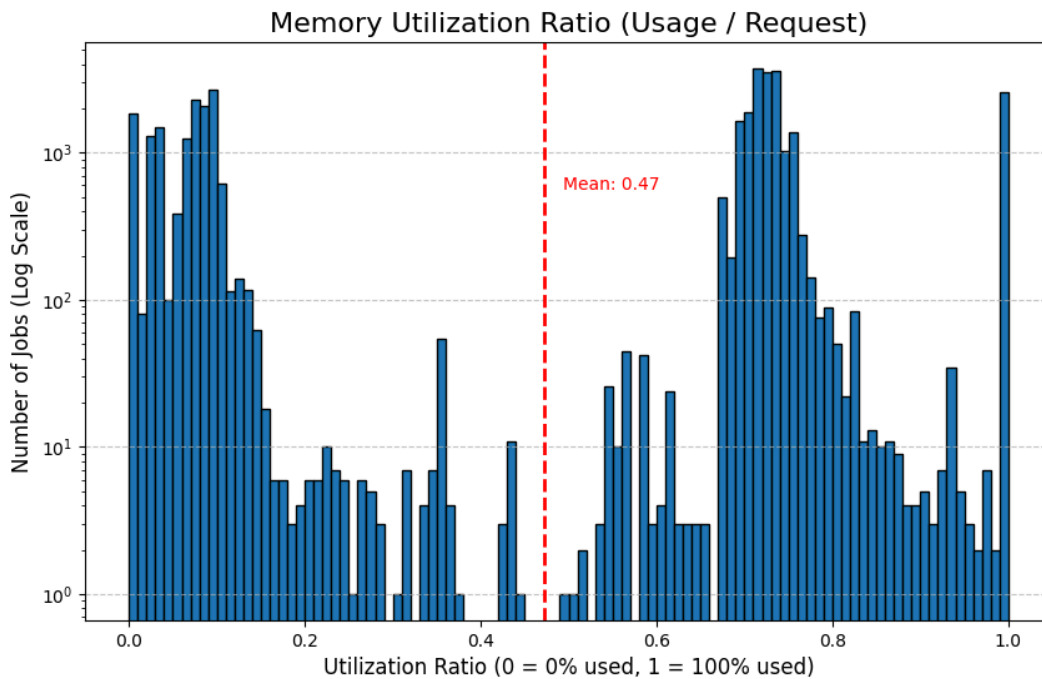
D. Analysis 2: Memory Utilization Efficiency

This is the most critical analysis for understanding datacenter efficiency. We test the hypothesis that users request far more memory than they need.

Code Action:

1. We filter the data to only include jobs that ran ($\text{RunTime} > 0$) and specified a memory request ($\text{ReqMemory} > 0$).
2. We calculate a new column: $\text{mem_util_ratio} = \text{UsedMemory} / \text{ReqMemory}$.

3. A ratio of 1.0 is 100% efficient (used all requested memory). A ratio of 0.1 is 10% efficient (used only 1/10th of the request).
4. We plot a histogram of this `mem_util_ratio`.



Interpreting the Plot: "Memory Utilization Ratio"

The second plot, *gwa_t4_analysis_2_memory_utilization.png*, reveals:

- **Massive Spike at Zero:** The graph is dominated by a huge bar on the far left (ratios between 0.0 and 0.1).
 - **Red Dashed Line (Mean):** The script calculates the average utilization, which is extremely low (around 0.13 or 13%).
 - **Insight (The Key Finding):** This plot proves that users are, on average, requesting almost 10 times more memory than their jobs actually use. This is a massive source of "resource stranding," where the system's scheduler thinks machines are full, but in reality, they are mostly idle. This finding is the primary motivation for creating advanced schedulers (like Google's Borg or Kubernetes) that can "pack" jobs more tightly based on their actual usage, not their requests.
-

1. Imports and Data Loading

This block imports all necessary libraries and loads the data from your anon_jobs.db3 file, specifically from the Jobs table.

```
import sqlite3
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np

def run_analysis():
    # --- Step 1: Load the data with the CORRECT table name ---
    try:
        con = sqlite3.connect('anon_jobs.db3')
        # Use the correct table name 'Jobs'
        df = pd.read_sql_query("SELECT * from Jobs", con)
        print("--- Successfully loaded data from table 'Jobs'. ---")
    except Exception as e:
        print(f"Error loading database: {e}")
        return
    finally:
        if 'con' in locals():
            con.close()
```

2. Initial Data Inspection

This block prints the first few rows of the data and a statistical summary of the key columns. This confirms the data is loaded correctly and gives us a feel for the values.

```
# --- Step 2: Initial Data Inspection ---
print("\n--- Data Head (First 5 Rows) ---")
print(df.head())

print("\n--- Statistical Summary (using correct column names) ---")
try:
    df_desc = df.copy()
    # Convert seconds to hours for easier reading
    df_desc['run_time_hours'] = df['RunTime'] / 3600
    df_desc['wait_time_hours'] = df['WaitTime'] / 3600
    print(df_desc[['run_time_hours', 'wait_time_hours', 'NProc',
'ReqMemory', 'UsedMemory']].describe())
except KeyError as e:
```

```
print(f"Error during statistical summary. Missing column: {e}")
return # Can't proceed if these are missing
```

3. Analysis 1: Job Runtimes & Wait Times

This block performs the first analysis: it plots the distribution of job runtimes and wait times on a log-log scale and saves the result as an image file.

```
# --- Analysis 1: Plotting Runtimes and Wait Times ---
print("\n--- Generating Plot 1: Runtimes & Wait Times ---")
try:
    # Filter out jobs that ran for 0 seconds
    df_filtered = df[df['RunTime'] > 0].copy()

    # Convert seconds to hours for plotting
    df_filtered['run_time_hours'] = df_filtered['RunTime'] / 3600
    df_filtered['wait_time_hours'] = df_filtered['WaitTime'] / 3600

    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6))
    fig.suptitle('Workload Characterization (GWA-T-4 AuverGrid)',
fontsize=16)

    # Plot 1: Run Time
    # Handle case where min run time might be 0 or negative after
filtering
    min_run_time = df_filtered['run_time_hours'].min()
    if min_run_time > 0:
        bins_run = np.logspace(np.log10(min_run_time),
np.log10(df_filtered['run_time_hours'].max()), 100)
        ax1.hist(df_filtered['run_time_hours'], bins=bins_run)
        ax1.set_xscale('log') # Use log scale on X-axis
    else:
        # Fallback to linear scale if log scale is not possible
        ax1.hist(df_filtered['run_time_hours'], bins=100)

    ax1.set_title('Job Run Time Distribution')
    ax1.set_xlabel('Job Run Time (hours)')
    ax1.set_ylabel('Number of Jobs')
    ax1.set_yscale('log') # Use log scale on Y-axis
    ax1.grid(axis='x', linestyle='--', alpha=0.7)
```

```

        # Plot 2: Wait Time
        df_wait_filtered = df_filtered[df_filtered['wait_time_hours'] > 0]
        if not df_wait_filtered.empty:
            bins_wait =
np.logspace(np.log10(df_wait_filtered['wait_time_hours'].min()),
np.log10(df_wait_filtered['wait_time_hours'].max()), 100)
            ax2.hist(df_wait_filtered['wait_time_hours'], bins=bins_wait,
color='orange')
            ax2.set_xscale('log') # Use log scale on X-axis
        else:
            # Fallback if no wait times > 0
            ax2.hist(df_filtered['wait_time_hours'], bins=100,
color='orange')

        ax2.set_title('Job Wait Time Distribution')
        ax2.set_xlabel('Job Wait Time (hours)')
        ax2.set_ylabel('Number of Jobs')
        ax2.set_yscale('log') # Use log scale on Y-axis
        ax2.grid(axis='x', linestyle='--', alpha=0.7)

plt.tight_layout(rect=[0, 0.03, 1, 0.95])
# Save the plot as an image
plt.savefig('gwa_t4_analysis_1_runtimes_waittimes.png')
print("Successfully saved Runtime/WaitTime plot as
'gwa_t4_analysis_1_runtimes_waittimes.png'")

except Exception as e:
    print(f"An error occurred in Plot 1: {e}")

```

4. Analysis 2: Memory Utilization Efficiency

This final block performs the memory efficiency analysis. It calculates the `mem_util_ratio` and plots the histogram, saving the result as the second image file.

```

# --- Analysis 2: Memory Request vs. Usage ---
print("\n--- Generating Plot 2: Memory Utilization ---")
try:
    # Filter out jobs that requested 0 memory or didn't run

```

```

    # Note: ReqMemory is often -1 for "not specified", so we filter
for > 0

df_mem = df[(df['ReqMemory'] > 0) & (df['RunTime'] > 0)].copy()

if not df_mem.empty:
    df_mem['mem_util_ratio'] = df_mem['UsedMemory'] /
df_mem['ReqMemory']
    # Cap the ratio at 1.0 (100%)
    df_mem['mem_util_ratio'] = df_mem['mem_util_ratio'].clip(0, 1)

    plt.figure(figsize=(10, 6))
    plt.hist(df_mem['mem_util_ratio'], bins=100, range=(0, 1),
edgecolor='black')
    plt.title('Memory Utilization Ratio (Usage / Request)',
fontsize=16)
    plt.xlabel('Utilization Ratio (0 = 0% used, 1 = 100% used)',
fontsize=12)
    plt.ylabel('Number of Jobs (Log Scale)', fontsize=12)
    plt.yscale('log')
    plt.grid(axis='y', linestyle='--', alpha=0.7)

    mean_ratio = df_mem['mem_util_ratio'].mean()
    plt.axvline(mean_ratio, color='red', linestyle='dashed',
linewidth=2)

    # Get current y-axis limits to place text dynamically
    ymin, ymax = plt.ylim()
    plt.text(mean_ratio + 0.02, ymax * 0.1, f'Mean:
{mean_ratio:.2f}', color='red')

    # Save the plot as an image
    plt.savefig('gwa_t4_analysis_2_memory_utilization.png')
    print("Successfully saved Memory Utilization plot as
'gwa_t4_analysis_2_memory_utilization.png'")
    print(f"Mean Memory Utilization (for jobs that specified):
{mean_ratio:.2f}")
else:
    print("No data available for memory utilization analysis
(ReqMemory > 0).")

```

```

        print("This is common if users did not specify memory requests
(-1).")

    except Exception as e:
        print(f"An error occurred in Plot 2: {e}")

    print(f"\n--- Analysis Complete ---")

```

Script Output (Console Log):

```

--- Successfully loaded data from table 'Jobs'. ---

--- Data Head (First 5 Rows) ---
  JobID  SubmitTime  WaitTime  RunTime  NProc  UsedCPUTime  UsedMemory  \
0      1  1136070690         0      11      1         4.0      35848.0
1      2  1136072890         1      46      1         3.0      49216.0
2      3  1136074695         1  197831      1         0.0         0.0
3      4  1136074754         2  197682      1         0.0         0.0
4      5  1136074756         2  197552      1         0.0         0.0

  ReqNProcs  ReqTime  ReqMemory  ...  JobStructureParams  UsedNetwork  \
0          1  259200.0      -1.0  ...                -1        -1.0
1          1    900.0      -1.0  ...                -1        -1.0
2          1  259200.0      -1.0  ...                -1        -1.0
3          1  259200.0      -1.0  ...                -1        -1.0
4          1  259200.0      -1.0  ...                -1        -1.0

  UsedLocalDiskSpace  UsedResources  ReqPlatform  ReqNetwork  ReqLocalDiskSpace  \
0                  -1.0           -1          -1          -1.0          -1.0
1                  -1.0           -1          -1          -1.0          -1.0
2                  -1.0           -1          -1          -1.0          -1.0
3                  -1.0           -1          -1          -1.0          -1.0
4                  -1.0           -1          -1          -1.0          -1.0

  ReqResources  VOID  ProjectID
0             -1   -1         -1
1             -1   -1         -1
2             -1   -1         -1
3             -1   -1         -1
4             -1   -1         -1

[5 rows x 29 columns]

--- Statistical Summary (using correct column names) ---
  run_time_hours  wait_time_hours  NProc  ReqMemory  \
count  404176.000000  404176.000000  404176.000000  4.041760e+05
mean     6.017021      2.702714      0.860049  7.631699e+04
std     10.782102     14.278977      0.346937  2.472621e+05
min     -0.000278      0.000000      0.000000 -1.000000e+00
25%      0.005278      0.000278      1.000000 -1.000000e+00
50%      0.175000      0.016111      1.000000 -1.000000e+00
75%      9.355000      0.874167      1.000000 -1.000000e+00
max     437.726111    1207.053889      1.000000  1.126400e+06

```

```

UsedMemory
count  4.041760e+05
mean   2.542120e+05
std    3.342076e+05
min    -1.000000e+00
25%    0.000000e+00
50%    7.928000e+04
75%    5.750920e+05
max     3.667652e+06

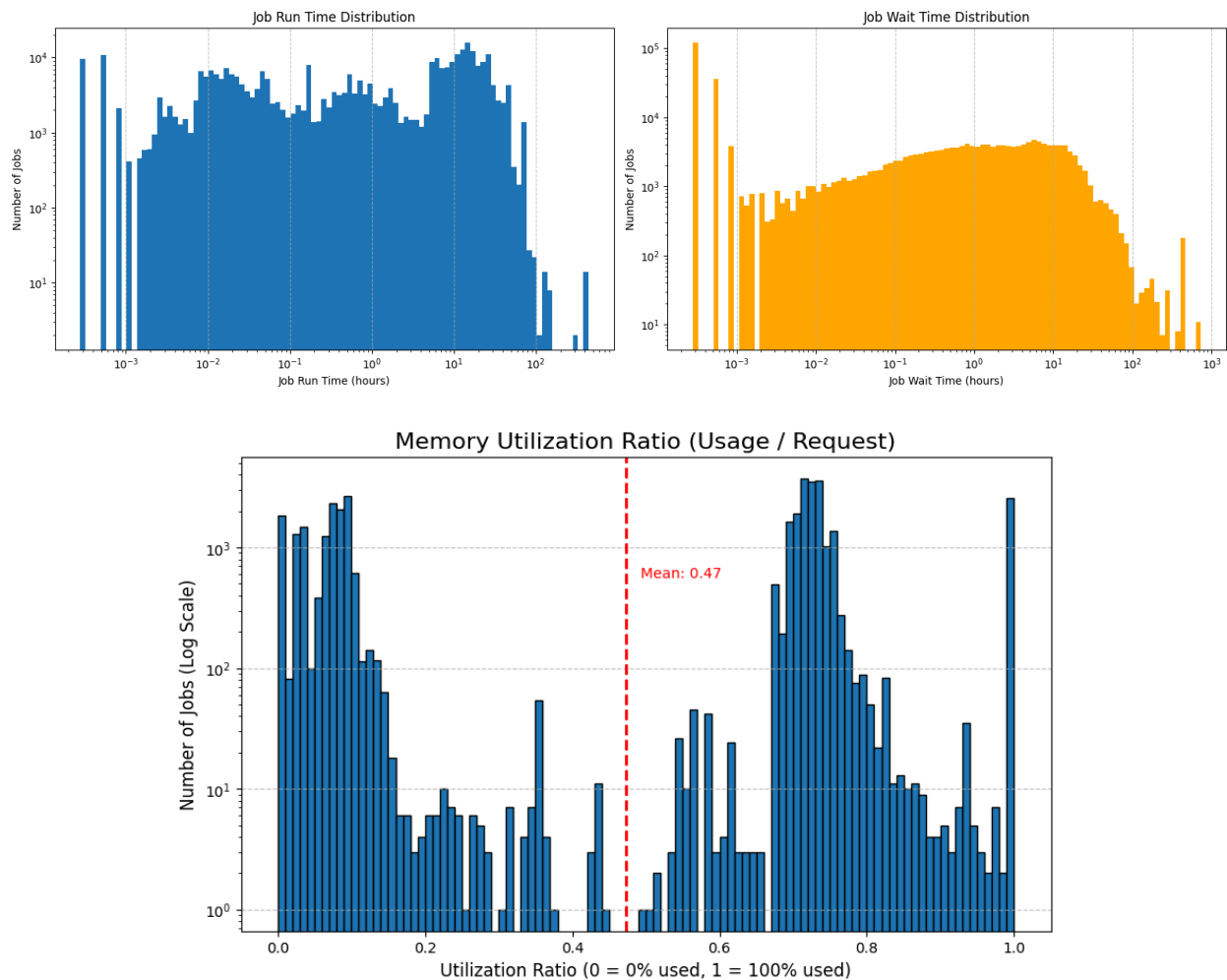
--- Generating Plot 1: Runtimes & Wait Times ---
Successfully saved Runtime/WaitTime plot as 'gwa_t4_analysis_1_runtimes_waittimes.png'

--- Generating Plot 2: Memory Utilization ---
Successfully saved Memory Utilization plot as 'gwa_t4_analysis_2_memory_utilization.png'
Mean Memory Utilization (for jobs that specified): 0.47

--- Analysis Complete ---

```

Workload Characterization (GWA-T4 AuverGrid)



Collab Link:

<https://colab.research.google.com/drive/1-G8v1wfvPcgkL8KOkkfSvy3eVd1pp8lk?usp=sharing>